

Pneumonia Detection Project

Technical Interview Presentation & Pipeline Guide

1. Executive Summary & Case Study

The Problem:

Pneumonia is a primary cause of infectious mortality globally. Diagnosing it requires visual evaluation of chest radiographs (X-rays). In primary care clinics and under-resourced hospitals, a severe shortage of certified radiologists leads to diagnostic delays (often exceeding 24-48 hours), causing critical treatment lags and increased patient risk.

The Market Gap:

Existing medical imaging AI software is heavily commercialized, closed-source, expensive, and demands specialized on-premise GPU workstations or complex enterprise cloud integrations. Small, regional clinics lack the budget and hardware to adopt these systems, leaving them without modern diagnostic assistance.

Our Solution:

We built a lightweight, CPU-optimized, browser-based diagnostic helper. It allows a general practitioner to drag-and-drop a patient's X-ray image and obtain a diagnostic prediction (Normal vs. Pneumonia) and a mathematical confidence score in under 100ms -- using basic office hardware.

Our Approach:

By fine-tuning a pre-trained ResNet-18 model on chest X-rays using Transfer Learning, we extracted robust feature representations without demanding massive clinical training data. We then converted the model to ONNX format and replaced PyTorch with ONNX Runtime to support fast, ultra-low-footprint inference on CPU.

Verified Project Performance Metrics:

- * **Validation Set Accuracy:** 81.25% (over 16-image split).
- * **Test Set Generalization Accuracy:** 82.05% (verified over 624 clinical test images).
- * **Inference Execution Speed:** Takes under 100ms per image on a single CPU thread via ONNX Runtime.
- * **RAM / Deployment Size:** Memory usage is cut to <70MB and slug deployment footprint is reduced by 400MB+.

2. Visual End-to-End Pipeline Flowchart

Use this diagram to show how data is structured and transformed as it moves from the web client through the preprocessing filters and ONNX Runtime inference engine to the final UI report:



3. Machine Learning Deep-Dive

Why Transfer Learning with ResNet-18?

Interviewers often ask why a specific model was chosen. ResNet-18 is a 18-layer Convolutional Neural Network that utilizes 'Skip Connections' (Residual blocks). Here is how you explain this choice:

* **Skip Connections:** These bypass layers and add inputs directly to outputs, preventing the vanishing gradient problem and allowing stable training.

* **Transfer Learning Advantages:** Medical datasets are often relatively small. Training from scratch causes overfitting. Starting with weights pre-trained on ImageNet allows the model to leverage edge and texture detection layers, requiring only fine-tuning on top-level classification structures.

* **Efficiency:** ResNet-18 is computationally lightweight (approx. 45MB file size), making it highly suitable for CPU-only server deployments like Render or local setups, while retaining excellent accuracy (82.05%).

ONNX Model Optimization:

To deploy this model efficiently without PyTorch's large library size (over 400MB CPU-only on disk):

1. Exported PyTorch weights to ONNX format using `torch.onnx.export` with input shapes of [1, 3, 224, 224].
2. Programmed PIL and NumPy-based preprocessing, matching PyTorch's transform pipeline exactly to eliminate torchvision dependencies.
3. Loaded the ONNX weights in the Flask backend using ONNX Runtime with `CPUExecutionProvider`, bypassing PyTorch completely.

4. Software Engineering & Deployment Design

A strong machine learning engineer must also demonstrate strong software engineering principles. Highlight these backend and architecture details in your discussion:

* **Flask Routing:** Exposes '/' for index, '/health' for container orchestration checks, and '/predict' for processing POST requests.

* **Robust Error Handling:** Wrapped the model loader and prediction pipeline in try-except blocks to return meaningful HTTP error templates (400 for bad uploads, 500 for inference failures) rather than crashing.

* **Security & File Management:** Filters file uploads using a secure whitelist of extensions (PNG, JPG, JPEG, GIF, BMP, WebP) and handles OS-independent file paths using python's `os.path` to prevent directory traversal exploits.

5. Top Interview Questions & Sample Answers

Q1: Explain your image preprocessing workflow and why it is necessary.

Answer strategy:

"Pre-trained models like ResNet-18 require input tensors to match the characteristics of the training dataset (ImageNet). First, I read the image and convert it to RGB. Next, I resize the image to 224x224 pixels. Finally, I convert it to a tensor and normalize it using ImageNet's mean [0.485, 0.456, 0.406] and standard deviation [0.229, 0.224, 0.225]. To optimize deployment, I implemented this pipeline in pure NumPy and Pillow, avoiding the heavyweight torchvision dependency in production."

Q2: How did you implement model inference in production under web server constraints?

Answer strategy:

"Instead of importing PyTorch, which has a 400MB+ disk footprint and consumes high memory, I converted the PyTorch model to ONNX and ran inference via ONNX Runtime. I initialize the ONNX InferenceSession once at Flask startup and cache it. This configuration uses under 70MB of RAM, executes in under 100ms on a single CPU core, and completely prevents OOM crashes on Render's 512MB RAM free tier."

Q3: What accuracy does this system achieve and how was it verified?

Answer strategy:

"To avoid misleading validation numbers, I evaluated the saved model against a large, unseen test set consisting of 624 clinical chest X-ray images. The model achieved a generalization accuracy of 82.05% (512 out of 624 images correctly classified). The model's loss is calculated using CrossEntropyLoss and optimized during training via the Adam optimizer. Having this verified evaluation proves the model generalizes reasonably well on real-world medical data."

Q4: What would you do to scale this system if it went from 10 users to 10,000 users?

Answer strategy:

"Currently, Flask handles requests synchronously. To scale this, I would:

1. Run the Flask application using a production WSGI server like Gunicorn with gevent or multiple worker processes.
2. Decouple inference from HTTP requests: instead of block-inferring inside Flask, I would put incoming image requests into a Celery queue backed by Redis, and run dedicated GPU worker instances to process the queue.
3. Containerize the application using Docker, and deploy it behind an Nginx load balancer on AWS ECS or EKS."